

Department Of Computer Studies

B. Sc. (CA & IT) Honours SEM-III

U12A2OOP- OBJECT ORIENTED CONCEPTS AND PROGRAMMING

Unit1: - Basic concept of Java

Procedural Oriented Programming	Object-Oriented Programming
In procedural programming, the program is divided into small parts called functions .	In object-oriented programming, the program is divided into small parts called objects .
Procedural programming follows a top-down approach .	Object-oriented programming follows a bottom-up approach .
There is no access specifier in procedural programming.	Object-oriented programming has access specifiers like private, public, protected, etc.
Adding new data and functions is not easy.	Adding new data and function is easy.
Procedural programming does not have any proper way of hiding data so it is less secure .	Object-oriented programming provides data hiding so it is more secure .
In procedural programming, overloading is not possible.	Overloading is possible in object-oriented programming.
In procedural programming, there is no concept of data hiding and inheritance.	In object-oriented programming, the concept of data hiding and inheritance is used.
Procedural programming is based on the unreal world .	Object-oriented programming is based on the real world .

Procedural Oriented Programming	Object-Oriented Programming
Examples: C, FORTRAN, Pascal, Basic, etc.	Examples: C++, Java, Python, C#, etc.

Introduction of Object Oriented Programming Concepts

- Object-Oriented Programming is a paradigm that provides many concepts, such as inheritance, data binding, polymorphism, etc.
- Simula is considered the first object-oriented programming language. The programming paradigm where everything is represented as an object is known as a truly object-oriented programming language.
- Smalltalk is considered the first truly object-oriented programming language.
- The popular object-oriented languages are Java, C#, PHP, Python, C++, etc.
- The main aim of object-oriented programming is to implement real-world entities, for example, object, classes, abstraction, inheritance, polymorphism, etc.
- **Object** means a real-world entity such as a pen, chair, table, computer, watch, etc. **Object-Oriented Programming** is a methodology or paradigm to design a program using classes and objects. It simplifies software development and maintenance by providing some concepts:
 - Object
 - Class
 - Inheritance
 - Polymorphism
 - Abstraction
 - Encapsulation

What is a class in Java

A Class is a blueprint or template used to create objects.

A **Class** is a user-defined data type that contains:

- Variables (called attributes or data members)

- Methods (called member functions)
- Constructors

It defines the properties and behaviors that objects created from the class will possess.

Real-Life Example of Class

Consider a **Student**.

Every student has:

Properties (Attributes)

- Roll Number
- Name
- Age
- Course

Behaviors (Methods)

- Study
- Attend Class
- Submit Assignment
- Take Exam

The class defines these common properties and behaviors.

Syntax of Class

```
class ClassName
```

```
{
```

```
    // Variables
```

```
    datatype variableName;
```

```
// Methods

returnType methodName()

{

    // code

}

}
```

Example of Class

```
class Student

{

    int rollNo;

    String name;

    void display()

    {

        System.out.println("Student Information");

    }

}
```

Student is the class name.

rollNo and name are instance variables.

display() is a method.

What is an Object in Java?

An **Object** is an instance of a class.

An Object is an instance of a class that contains actual values for the attributes defined in the class.

When memory is allocated to a class at runtime, an object is created.

Real-Life Example of Object

If **Student** is a class:

THEN

Rahul

Priya

Neha are objects of the Student class.

Example:

Class	Objects
Student	Rahul, Priya
Car	BMW, Audi
Mobile	Samsung, iPhone
Employee	Raj, Rahul

Syntax of Object Creation

```
ClassName objectName = new ClassName();
```

Example

```
Student s1 = new Student();
```

Where:

- Student → Class Name
- s1 → Object Reference Variable
- new → Keyword used to allocate memory
- Student() → Constructor

Accessing Class Members Using Object

```
objectName.variableName;  
objectName.methodName();
```

Example:

```
s1.rollNo = 101;
```

```
s1.name = "Rahul";
```

```
s1.display();
```

State, Behavior, and Identity of an Object

Every object has three characteristics:

1. State

Represents the current values of attributes.

Example:

Name=Rahul

Age = 20

2. Behavior

Represents actions performed by the object.

Example:

```
study()
writeExam()
```

3. Identity

Uniquely identifies an object in memory.

Example:

Student@15db9742

Example: Bank Account Program – Bank.java

```
class BankAccount

{

    String accountHolder;

    double balance;

    void display()

    {

        System.out.println("Account Holder: " + accountHolder);

        System.out.println("Balance: " + balance);

    }

}

public class Main

{

    public static void main(String args[])
```

```
{  
  
    BankAccount acc1 = new BankAccount();  
  
    acc1.accountHolder = "Ramesh";  
  
    acc1.balance = 50000;  
  
    acc1.display();  
  
}  
  
}
```

Encapsulation

Encapsulation is one of the four fundamental principles of Object-Oriented Programming (OOP) in Java, along with:

1. Encapsulation
 2. Inheritance
 3. Polymorphism
 4. Abstraction
- Encapsulation is the process of binding data (variables) and methods (functions) together into a single unit called a class and restricting direct access to the data from outside the class.
 - It helps in protecting the internal state of an object and allows controlled access through public methods.
 - Encapsulation is the mechanism of wrapping data members and methods into a single unit and hiding the implementation details by restricting direct access to the data.

Real-Life Example of Encapsulation

Consider an ATM Machine.


```
}
```

```
class Cat extends Animal {
```

```
    void meow() {
```

```
        System.out.println("Cat meows");
```

```
    }
```

```
}
```

```
public class hiera {
```

```
    public static void main(String[] args) {
```

```
        Dog d = new Dog();
```

```
        Cat c = new Cat();
```

```
        System.out.println("Dog Object:");
```

```
        d.eat();
```

```
        d.bark();
```

```
        System.out.println();
```

```
        System.out.println("Cat Object:");
```

```
        c.eat();
```

```
        c.meow();
```

```
    }
```

```
}
```

4. Multiple Inheritance (Not Supported with Classes)

Java does not support multiple inheritance using classes to avoid ambiguity problems.

5. Hybrid Inheritance

Combination of multiple inheritance types.

Java supports hybrid inheritance only through interfaces.

Polymorphism

Polymorphism is the ability of an object, method, or reference variable to take multiple forms and behave differently in different situations.

Polymorphism means "one object or method can have many forms or behaviors."

For example:

A person can have different roles:

- Teacher in school
- Parent at home
- Customer in a store

Although the person is the same, the behavior changes according to the situation.

Similarly, in Java:

Same method name Different behavior

Why is Polymorphism Needed?

Polymorphism provides:

- Code reusability
- Flexibility
- Extensibility
- Better maintainability

- Dynamic method execution

Without polymorphism, programmers would need to create separate methods for every operation.

Types of Polymorphism in Java

Java supports two types of polymorphism:

1. Compile-Time Polymorphism

Also called:

- Static Polymorphism
- Early Binding
- Method Overloading

2. Run-Time Polymorphism

Also called:

- Dynamic Polymorphism
- Late Binding
- Method Overriding

1. Compile-Time Polymorphism

Definition

Compile-time polymorphism occurs when the method to be executed is determined by the compiler during compilation.

This is achieved through:

Method Overloading

Multiple methods having the same name but different parameter lists.

Example of Method Overloading

```
class Calculator {

    int add(int a, int b) {

        return a + b;

    }

    int add(int a, int b, int c) {

        return a + b + c;

    }

    double add(double a, double b) {

        return a + b;

    }

}

public class Main {

    public static void main(String[] args) {

        Calculator c = new Calculator();

        System.out.println(c.add(10,20));

        System.out.println(c.add(10,20,30));

        System.out.println(c.add(10.5,20.5));

    }

}
```

2. Run-Time Polymorphism

Definition

Run-time polymorphism occurs when the method to be executed is determined during program execution.

This is achieved using:

Method Overriding

A subclass provides its own implementation of a method already defined in the parent class.

Example of Method Overriding

```
class Bank {  
  
    void interestRate() {  
  
        System.out.println("Bank Interest Rate");  
  
    }  
  
}  
  
class SBI extends Bank {  
  
    void interestRate() {  
  
        System.out.println("SBI Interest Rate is 7%");  
  
    }  
  
}  
  
class HDFC extends Bank {  
  
    void interestRate() {
```

```
        System.out.println("HDFC Interest Rate is 8%");

    }

}

public class overriding {

    public static void main(String args[]) {

        Bank b;

        b = new SBI();

        b.interestRate();

        b = new HDFC();

        b.interestRate();

    }

}
```

Java Abstraction

Abstraction is a process of hiding the implementation details from the user, only the functionality will be provided to the user.

In other words, the user will have the information on what the object does instead of how it does it.

In Java programming, abstraction is achieved using **Abstract classes and interfaces**.

Abstraction is used to **hide unnecessary implementation details** and show only the **essential features** of an object to the user.

Ways to achieve Abstraction

There are two ways to achieve abstraction in java

1. Abstract class
2. Interface

Abstract class in Java

A class which is declared as abstract is known as an **abstract class**.

It can have abstract and non-abstract methods. It needs to be extended and its method implemented. It cannot be instantiated.

Points to Remember

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have constructors and static methods also.
- It can have final methods which will force the subclass not to change the body of the method.

Example of abstract class

Syntax

```
abstract class ClassName {  
  
    abstract void methodName();  
  
}
```

Example of Abstract Class

```
abstract class Animal {  
  
    abstract void sound();  
  
}
```

```
class Dog extends Animal {  
  
    void sound() {  
  
        System.out.println("Dog barks");  
  
    }  
  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Dog d = new Dog();  
  
        d.sound();  
  
    }  
  
}
```

Output

Dog barks

2. Interface

An interface provides **100% abstraction**.

It contains method declarations without implementation.

Example Using Interface

```
interface Shape {  
  
    void draw();  
  
}
```



```
}
```

```
class Circle implements Shape {
```

```
    public void draw() {
```

```
        System.out.println("Drawing Circle");
```

```
    }
```

```
}
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Shape s = new Circle();
```

```
        s.draw();
```

```
    }
```

```
}
```

Abstract Method in Java

Abstract Methods

If you want a class to contain a particular method but you want the actual implementation of that method to be determined by child classes, you can declare the method in the parent class as an abstract.

- **abstract** keyword is used to declare the method as abstract.
- You have to place the **abstract** keyword before the method name in the method declaration.
- An abstract method contains a method signature, but no method body.
- Instead of curly braces, an abstract method will have a semicolon (;) at the end.

Following is an example of the abstract method.

Example of Abstract class that has an abstract method

In this example, Bike is an abstract class that contains only one abstract method run. Its implementation is provided by the Honda class.

```
1. abstract class Bike{
2.   abstract void run();
3. }
4. class Honda4 extends Bike{
5.   void run(){System.out.println("running safely");}
6.   public static void main(String args[]){
7.     Bike obj = new Honda4();
8.     obj.run();
9.   }
10. }
```

Test it Now

running safely

Java Example

```
abstract class Television {
```

```
    abstract void powerOn();
```

```
    abstract void changeChannel();
```

```
}
```

```
class SmartTV extends Television {
```

```
    void powerOn() {
```

```
        System.out.println("TV is turned ON");
```

```
    }
```

```
    void changeChannel() {
```

```
        System.out.println("Channel changed");
```

```
    }
```

```
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Television tv = new SmartTV();  
  
        tv.powerOn();  
  
        tv.changeChannel();  
  
    }  
  
}
```

Output

TV is turned ON

Channel changed

Data Hiding

Data Hiding is the technique of hiding the implementation details and internal data of a class from outside classes by declaring variables as private and providing public methods to access or modify them.

In simple words, data hiding protects the data of an object from unauthorized access and modification.

Need for Data Hiding

Suppose a student's age is stored in a variable.

Without data hiding:

```
student.age = -20;
```

This is invalid because age cannot be negative.

With data hiding:

```
student.setAge(-20);
```

The setter method can check whether the value is valid before storing it.

Thus, data hiding helps maintain **data integrity and security**.

How Data Hiding is Achieved in Java

Java uses **Access Modifiers** to implement data hiding.

The most commonly used modifier is:

Private

Private variables cannot be accessed directly outside the class.

Access is provided using:

- Getter methods
- Setter methods

Example Without Data Hiding

```
class Student
```

```
{
```

```
    public int marks;
```

```
}
```

```
public class Main
```

```
{
```

```
public static void main(String args[])

{

    Student s = new Student();

    s.marks = -100;

    System.out.println(s.marks);

}

}
```

Output

-100

This is incorrect because marks cannot be negative.

Example With Data Hiding

```
class Student

{

    private int marks;

    public void setMarks(int m)

    {

        if(m >= 0 && m <= 100)

        {

            marks = m;
```

```
}
```

```
else
```

```
{
```

```
    System.out.println("Invalid Marks");
```

```
}
```

```
}
```

```
public int getMarks()
```

```
{
```

```
    return marks;
```

```
}
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String args[])
```

```
{
```

```
        Student s = new Student();
```

```
        s.setMarks(85);
```

```
        System.out.println("Marks = " + s.getMarks());
```

```
}
```

```
}
```

Output

Marks = 85

Getter and Setter Methods

Getter Method

A getter method is used to read or retrieve private data.

Example:

```
public int getMarks()
```

```
{
```

```
    return marks;
```

```
}
```

Setter Method

A setter method is used to update private data.

Example:

```
public void setMarks(int m)
```

```
{
```

```
    marks = m;
```

```
}
```

Real-World Example: Bank Account

Consider a bank account.

The balance should not be directly modified by users.

Program

```
class BankAccount

{

    private double balance;

    public void deposit(double amount)

    {

        if(amount > 0)

        {

            balance += amount;

        }

    }

    public void withdraw(double amount)

    {

        if(amount <= balance)

        {

            balance -= amount;

        }

    }

}
```



```
    else

    {

        System.out.println("Insufficient Balance");

    }

}

public double getBalance()

{

    return balance;

}

}

public class Main

{

    public static void main(String args[])

    {

        BankAccount acc = new BankAccount();

        acc.deposit(10000);

        acc.withdraw(3000);

        System.out.println("Balance = " + acc.getBalance());

    }

}
```

```
}
```

Output

Balance = 7000.0

Dynamic Binding in Java

Definition

Dynamic Binding (also called **Late Binding** or **Runtime Binding**) is the process of linking a method call to its method implementation at **runtime rather than compile time**.

In dynamic binding, the method that gets executed depends on the **actual object type**, not on the **reference type**.

Java performs dynamic binding for **overridden methods**.

Simple Definition

Dynamic Binding means **deciding which method to execute during program execution (runtime)**.

Why Dynamic Binding is Needed?

Suppose a parent class reference points to a child class object.

```
Animal a = new Dog();
```

When:

```
a.sound();
```

is executed, Java determines at runtime whether to call:

- `Animal.sound()`
- `Dog.sound()`

The actual object is Dog, so Java calls:

```
Dog.sound();
```

This decision is made during runtime.

Example of Dynamic Binding

```
class Animal
```

```
{
```

```
    void sound()
```

```
{
```

```
    System.out.println("Animal makes sound");
```

```
}
```

```
}
```

```
class Dog extends Animal
```

```
{
```

```
    void sound()
```

```
{
```

```
    System.out.println("Dog barks");
```

```
}
```

```
}
```

```
public class Test
```

```
{  
  
    public static void main(String args[])  
  
    {  
  
        Animal a = new Dog();  
  
        a.sound();  
  
    }  
  
}
```

Output

Dog barks

Another Example

```
class Shape  
  
{  
  
    void draw()  
  
    {  
  
        System.out.println("Drawing Shape");  
  
    }  
  
}  
  
class Circle extends Shape  
  
{
```

```
void draw()

{

    System.out.println("Drawing Circle");

}

}

class Rectangle extends Shape

{

    void draw()

    {

        System.out.println("Drawing Rectangle");

    }

}

public class Demo

{

    public static void main(String args[])

    {

        Shape s;

        s = new Circle();

        s.draw();

    }

}
```

```
s = new Rectangle();  
  
s.draw();  
  
}  
  
}
```

Output

DrawingCircle

Drawing Rectangle

Message Communication in Java

Definition

Message Communication is the process by which one object communicates with another object by calling its methods.

In Object-Oriented Programming, objects interact by sending messages to each other.

Simple Definition

Message Communication is the interaction between objects through method calls.

Components of Message Communication

A message consists of three components:

1. Object Name

The object receiving the message.

2. Method Name

The action requested.

3. Arguments

Optional values passed to the method.

Syntax

```
objectName.methodName(arguments);
```

Example:

```
student.display();
```

Example of Message Communication

```
class Student
```

```
{  
  
    void display()  
  
    {  
  
        System.out.println("Student Information");  
  
    }  
  
}
```

```
public class Test
```

```
{  
  
    public static void main(String args[])  
  
    {  
  
        Student s = new Student();  
  
    }  
  
}
```

```
s.display();  
  
}  
  
}
```

Output

Student Information

The object s sends a message:

display()

to the Student object.

Advantages of OOP in Java

1. Code Reusability

One of the biggest advantages of OOP is code reusability through inheritance.

Instead of writing the same code multiple times, a new class can inherit the properties and methods of an existing class.

2. Data Security through Encapsulation

Encapsulation hides internal data from unauthorized access and provides controlled access using getter and setter methods.

3. Easy Maintenance and Modification

Since programs are divided into classes and objects, modifying one part of the application does not affect the entire system.

Example

If the Employee class changes, other unrelated classes remain unaffected.

Benefit

- Simplifies debugging.
- Easier updates and maintenance.

4. Supports Modularity

Large programs can be divided into smaller independent modules called classes.

Example

A banking system may contain:

- Account class
- Customer class
- Transaction class
- Loan class

Benefit

- Improves program organization.
- Facilitates teamwork.

5. Supports Polymorphism

Polymorphism allows one interface to be used for multiple implementations.

6. Supports Abstraction

Abstraction hides implementation details and shows only essential features to users.

Example

When driving a car, users use the steering wheel and pedals without knowing the engine's internal working.

7. Better Problem Solving Approach

OOP models software according to real-world objects and interactions.

Example

A university management system contains:

- Student objects
- Faculty objects
- Course objects
- Examination objects

Benefit

- Easier to understand and design.

8. Improves Software Quality

OOP encourages:

- Reusability
- Reliability
- Maintainability
- Scalability

Benefit

- Produces high-quality software systems.

9. Easier Testing and Debugging

Since programs are divided into classes, errors can be isolated easily.

Benefit

- Faster debugging.
- Reduced testing time.

10. Supports Scalability

New features can be added without rewriting existing code.

Example

Adding a new payment method in an e-commerce application.

Benefit

- Supports future expansion.

Applications of OOP in Java

Object-Oriented Programming is widely used in modern software development.

1. Banking Systems

2. Hospital Management Systems

3. E-Commerce Applications

4. Library Management Systems

5. Student Management Systems

6. Online Reservation Systems

7. Game Development

8. Mobile Application Development

9. Enterprise Applications

10. Scientific and Research Applications

